

Silicon Jackets Physical Design Onboarding Project

Name: Saksham Bansal

Discord ID: bansalsaksham

GT Email: sbansal91@gatech.edu

1. Introduction to Physical Design: OpenLane Colab Notebook

In this section, you will follow along with the Physical Design flow to generate physical IC layouts from RTL using OpenLane.

OpenLane is an open-source toolchain for physical design. At SiliconJackets, we use the Cadence suite, not OpenLane. However, following along with this flow is a good way to get an idea of what the Physical Design team does.

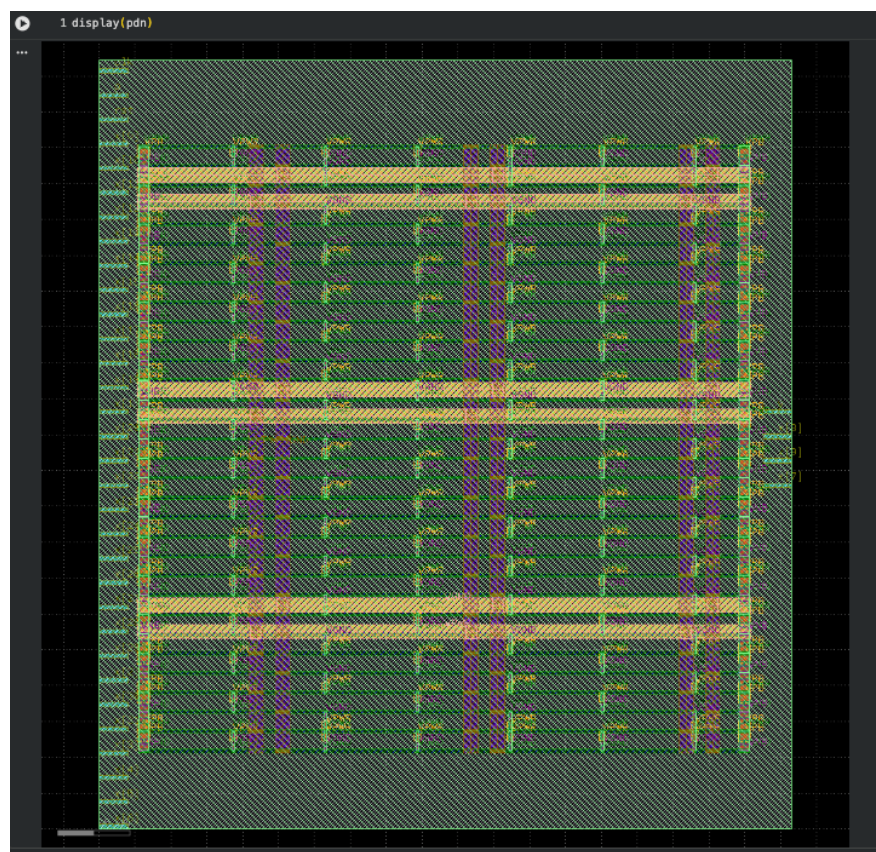
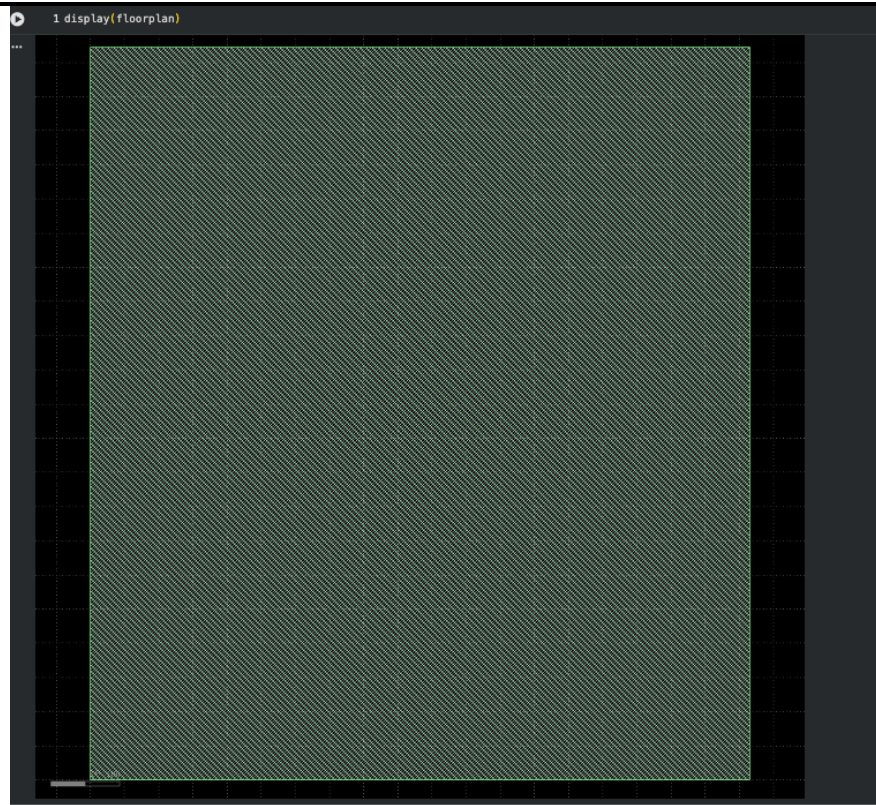
i. Follow [this link](#) to create a copy of the OpenLane Colab Notebook.

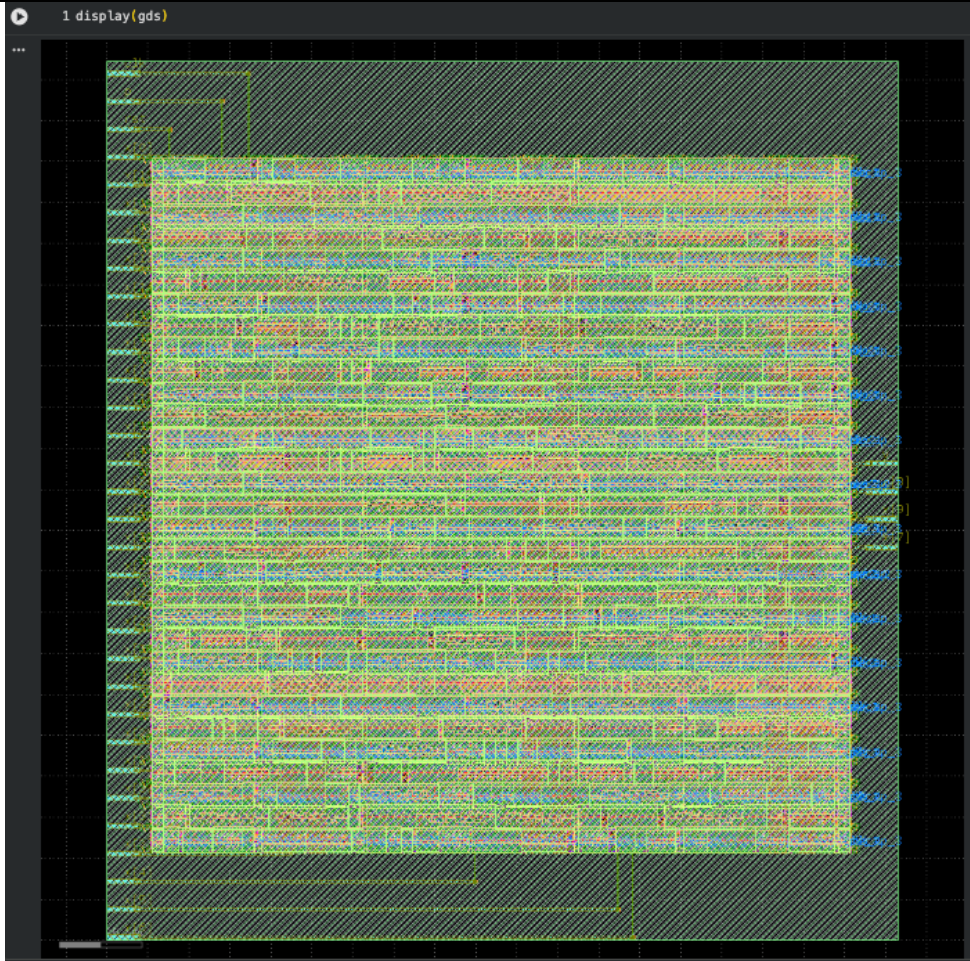
For in-depth information about the OpenLane flow, refer to the [OpenLane reference manual](#).

ii. Run all the steps in the OpenLane Colab Notebook. Make sure to read each part thoroughly so you understand what the Notebook is actually doing.

iii. Take screenshots of your Notebook after completing each of the following steps and paste them into the answer box on the next page.

- Floorplan
- PDN Generation
- GDS Streamout





iv. Describe in your own words what the following flow steps do.

a. Floorplanning

Floorplanning is the initial physical setup of the chip. It determines the overall dimensions (height and width) of the final silicon die and establishes the core placement grid (rows and sites) where standard logic cells will eventually be placed. This step also involves placing large macro blocks and defining the locations for the input/output (I/O) pins.

b. Clock Tree Synthesis

Clock Tree Synthesis is the process of routing the primary clock signal to every sequential element (like flip-flops) in the design simultaneously. Because physical wires introduce delays, CTS inserts a balanced, hierarchical network of clock buffers across the chip to minimize "clock skew", ensuring the clock arrives at every register at the exact same time.

c. Global Route

Global routing is a high-level planning stage for the chip's wiring. Instead of drawing the exact physical metal wires, the routing tool divides the chip into a coarse grid and creates "routing guides." These guides define the approximate paths the wires will take to connect logic gates, allowing the tool to avoid areas of high congestion before committing to the exact metal tracks in the subsequent Detailed Routing phase.

d. Signoff Static Timing Analysis (Post-PnR STA)

Signoff STA is the final, highly accurate timing verification of the completed physical layout. It takes the final routed netlist and the exact resistance and capacitance of the physical metal wires to perform a rigorous mathematical check. This ensures that no signals arrive too late (setup violations) or too early (hold violations), guaranteeing the chip will actually function at its intended clock speed after it is manufactured.

2. Clocking a Sequential Circuit

In this section, you'll get an introduction to critical paths, setup times, and identifying viable clock frequencies.

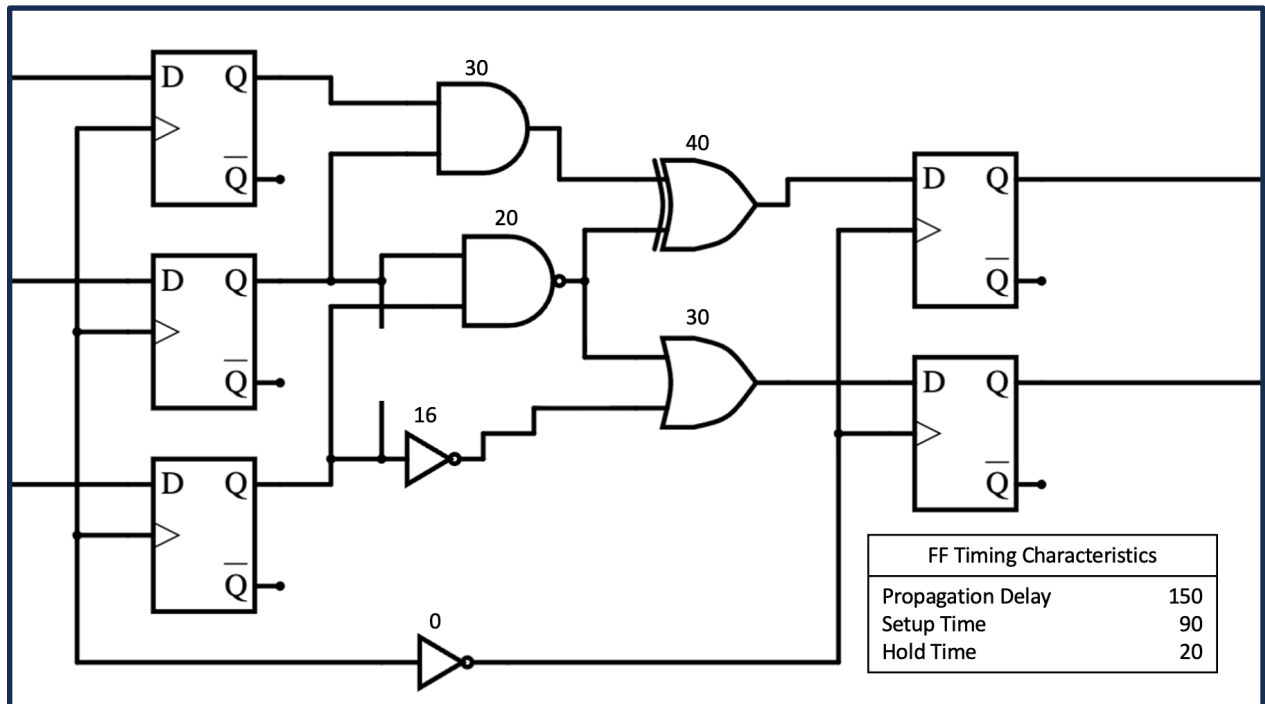
Performing timing analysis on an IC with millions of transistors requires many of the same essential skills as performing timing analysis on a breadboard circuit diagram — just on a much different scale.

This section requires knowledge of some Static Timing Analysis concepts you may not be familiar with.

To learn about Setup and Hold times, see [this resource](#).

To learn about Static Timing Analysis, see [this resource](#) or [this more comprehensive resource](#).

Consider the following Sequential Circuit.



i. Identify the worst case timing path in this circuit (e.g FF -> INV -> AND -> OR -> FF). Calculate the worst negative slack (WNS) if the clock period here is 500 ps. Assume all units are picoseconds.

- **Critical Path:** Top FF -> AND -> XOR -> Top Right FF (or Mid FF -> AND -> XOR -> Top Right FF).
- **WNS Calculation:** Data Arrival Time = 150ps (T_{cq}) + 70ps (Logic) = 220ps.
Due to the clock inverter, this is a half-cycle path with a capture edge at 250ps.
Required Time = 250ps - 90ps (T_{setup}) = 160ps.
Slack = 160ps - 220ps = **-60 ps**.

li. Given this critical path, what is the maximum viable clock frequency?

Max Frequency: Setting Slack to 0 for the half-cycle path:

$$T_{period}/2 - T_{setup} = T_{cq} + T_{logic}$$

$$T_{period}/2 - 90 = 220$$

$$T_{period} = 620 \text{ ps.}$$

The maximum viable clock frequency is **1.61 GHz**.

iii. Explain the concepts of setup slack and hold time. Why is a negative slack value problematic?

- **Setup Slack:** This is the difference between the time a signal is required to arrive (T_{req}) and the time it actually arrives (T_{arr}) at a sequential element. It represents the margin of safety before a setup violation occurs.
- **Hold Time:** This is the minimum amount of time that a data signal must remain stable *after* the active clock edge to be reliably captured by the flip-flop.
- **Why negative slack is problematic:** Negative slack means the data signal did not reach the flip-flop and stabilize before the setup window closed. If this happens, the flip-flop may capture old data, capture a transitional voltage, or enter a metastable state, causing immediate functional failure of the chip.

3. Python Timing Report Analysis

In this section, you'll get an introduction to the analysis of timing reports generated by our EDA tools using Python.

We've provided a sample timing report (`full_paths.max.rpt`) for you to analyze.

The report is too large to analyze by hand, so you'll have to write a Python script to help parse the file data. You can use the provided `template.py` if you'd like.

[This guide on File I/O](#) and [this guide on String processing](#) might be good places to start if you don't have any Python experience.

i. How many paths violated timing constraints (i.e., have negative slack)?

50 paths violated timing constraints.

ii. What is the average depth (or number of gates) of paths with the smallest positive slack?

Hint: In this context, "smallest positive slack" refers to paths that *are* passing timing, but just barely.

Answering this question might involve creating a script that extracts tuples containing each path's slack and gate depth, then sorting the tuples by slack.

The smallest positive slack is **0.0 ns**, and the average logic depth for those paths is **7.0 gates**.

iii. What clock speed was the design run at when this timing report was generated?

Does the report allow us to calculate a better clock speed for this design? If so, please calculate it.

The design was run at **1.0 GHz** (1000 MHz), which is indicated by the clock `clk` (rise edge) 1.00 line in the required time calculations, meaning the period is set to 1.00 ns.

Better clock speed: From a timing perspective, there is **no room** to push for higher performance. Because there are already 50 paths with negative slack (failing paths), the chip cannot even run reliably at the current 1.0 GHz. We would need to calculate a *slower* clock speed (e.g., subtracting the Worst Negative Slack from the 1.00 ns period) to find a viable operating frequency.

4. Flow Debugging

If you haven't already, run the flow according to the directions in RTL2GDS/README.md. You may notice that the flow fails! Read the flowtool.log file (or the output directly to your terminal; its the same thing) to find the source of the error.

Unless you know what to look for, the log does a pretty poor job telling you what's wrong. To deduce the source of the error, look at the name of the flow step that fails. You should be able to find this flow setup in scripts/flow.yaml. Flow steps in flow.yaml correspond to actual Tcl code in certain places in the scripts directory.

For example, run_syn_generic in flow.yaml corresponds to a flow step in genus_steps.tcl that runs a single command (this is not the failing flow step):

```
#####  
# STEP run_syn_generic  
#####  
create_flow_step -name run_syn_generic -owner cadence {  
  #- Synthesize to generic gates  
  syn_generic  
}
```

The failing flow step is referenced in flow.yaml, but it is *defined* somewhere else. You'll have to edit the definition of the failing flow step in another file to resolve this issue.

Once you've fixed it, rerun the flow, and it should run to completion.

No submission required for this section.

5. Tcl Procedures

This section focuses on the Tool Command Language (Tcl), which we use to write scripts that automate aspects of running the flow. Use the template file `scripts/onboarding.tcl` for this section.

*You need to run the flow **to completion** before answering problems in this section. See the README **inside the flowtool directory** for instructions on running the flow and complete part 4 before attempting this part.*

You will also have to use Cadence Innovus. For information on using Cadence Innovus, read `Using-Innovus.md` in `docs/`

i. Navigate to the *design/scripts* directory. Choose a *.tcl* script in this directory and explain what it does.

I reviewed the scripts/floorplan.tcl script. This script automates the physical floorplanning stage of the design. It defines the core dimensions (the overall size of the chip), specifies the exact X and Y coordinate locations to place the large hard macros (like our two SRAM blocks at coordinates 50, 50 and 50, 800), and adds placement blockages or halos around those macros to ensure standard cells aren't placed too close to them, preventing routing congestion.

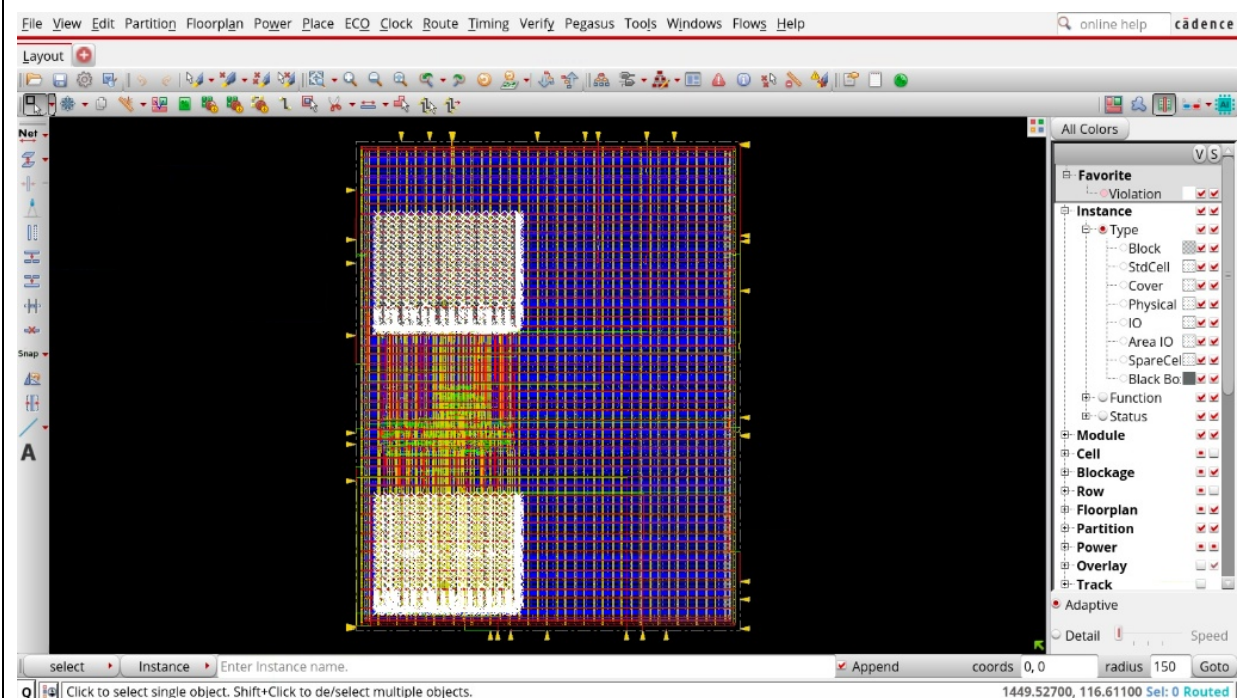
ii. We want to highlight an SRAM block in our design.

a. Consult the Cadence Support website to identify the Innovus commands to dim the foreground and highlight an object.

Use these commands in the Innovus shell to highlight an SRAM block and paste a screenshot in the box below.

To dim the workspace and highlight an object using Cadence Common UI (CUI) commands in the Innovus shell, I used the following commands:

1. `deselect_obj -all` (clears existing selections)
2. `select_obj [get_db insts *sram_A*]` (selects and highlights the SRAM block)
3. `zoomSelected` (zooms the camera to the highlighted block)



b. Let's automate that process. Complete the Tcl proc in scripts/onboarding.tcl to take an SRAM's name as an input and invoke these two commands to highlight it in Innovus.

Paste or screenshot your Tcl proc code here, and show the file being sourced in the Innovus shell.

```
proc highlight_block {inst} {  
    # Clear previous selections  
  
    deselect_obj -all  
  
    # Get the instance pointer from the database  
  
    set target_inst [get_db insts $inst]  
  
    if {$target_inst != ""} {  
        # Highlight the object and zoom to it  
  
        select_obj $target_inst  
  
        zoomSelected  
  
        puts "Successfully highlighted and zoomed to: $inst"  
    } else {  
        puts "Error: Could not find instance matching '$inst'. "  
    }  
}
```

sbansal91@ece-rschsrv:~/Physical-Design-Onboarding-S26/RTL2GDS/flowtool

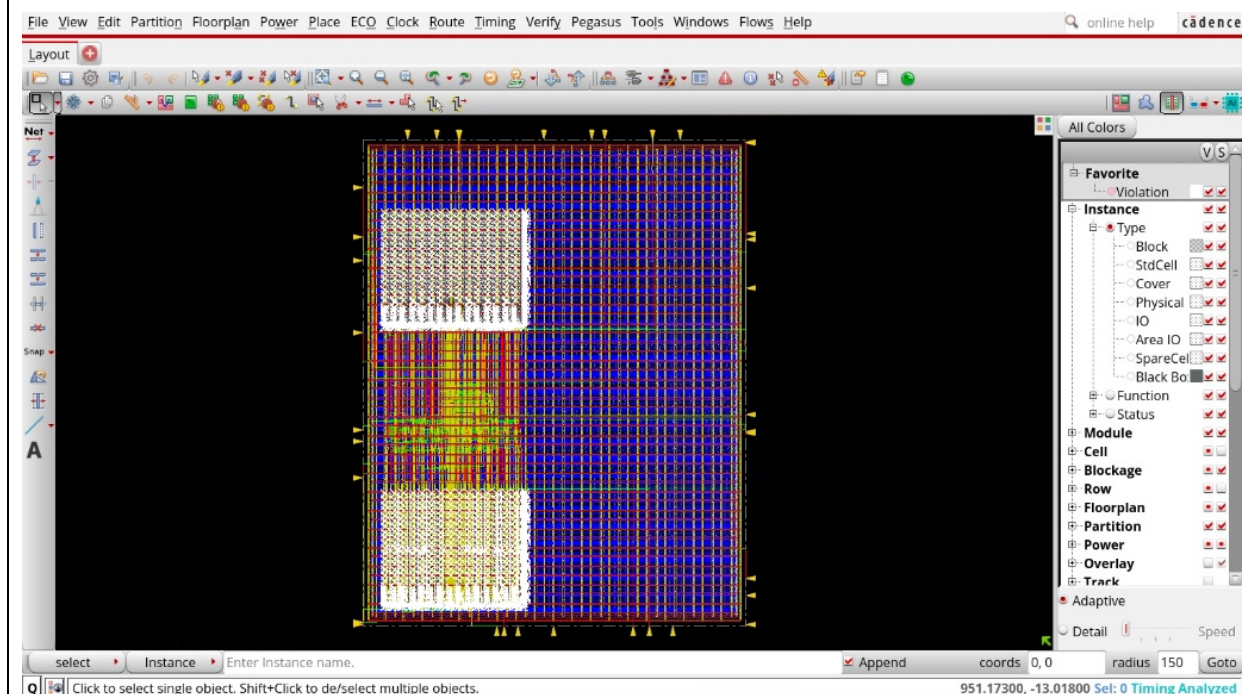
```
invalid command name "zoom_to"  
innovus 5> source scripts/onboarding.tcl  
innovus 6> highlight_block *sram_A*  
Successfully highlighted and zoomed to: *sram_A*  
innovus 7> dump_fanouts  
Clock Buffer: postroute_FE_PHC354_read_start_addr_4 | Type: CLKBUF2 | Fanout: 1  
Clock Buffer: postroute_FE_PHC347_u_resbuf_internal_buffer_20 | Type: CLKBUF2 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC345_u_resbuf_internal_buffer_21 | Type: CLKBUF4 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC343_buffer_word_42 | Type: CLKBUF2 | Fanout: 1  
Clock Buffer: postroute_FE_PHC342_u_resbuf_internal_buffer_5 | Type: CLKBUF4 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC335_u_resbuf_internal_buffer_14 | Type: CLKBUF4 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC332_u_resbuf_internal_buffer_24 | Type: CLKBUF4 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC321_u_resbuf_internal_buffer_15 | Type: CLKBUF2 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC320_u_resbuf_internal_buffer_4 | Type: CLKBUF4 |  
Fanout: 2  
Clock Buffer: postroute_FE_PHC319_u_resbuf_internal_buffer_11 | Type: CLKBUF2 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC317_u_resbuf_internal_buffer_16 | Type: CLKBUF2 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC313_u_resbuf_internal_buffer_9 | Type: CLKBUF2 |  
Fanout: 1  
Clock Buffer: postroute_FE_PHC308_u_resbuf_internal_buffer_1 | Type: CLKBUF2 |
```

iii. We can also use Tcl to highlight paths in our design. This is especially useful for highlighting paths that fail timing to help us decide how to optimize our layout to improve timing.

Find and use Innovus commands that read a timing debug report and highlight the paths in the GUI. Paste a screenshot of the GUI showing failed paths in the box below.

To highlight failing timing paths in the GUI, I generated a binary timing report and loaded it into the Innovus Timing Debugger. Innovus 25.11 requires the binary format for timing debug reports. I used the following commands in the Innovus console:

1. `report_timing -output_format binary > my_timing_report.bin`
2. `load_timing_debug_report my_timing_report.bin`
3. `highlight_timing_report -file my_timing_report.bin -path 1`



iv. Let's automate this too. Complete the Tcl proc that takes in a path number as an input and highlights in the GUI.

Paste or screenshot your Tclproc code here, as well as the file being sourced in the Innovus shell.

```
proc highlight_path {path_num} {  
  
    # Clear existing selections in the GUI
```

```

deselect_obj -all

# Generate the report using the required binary format
report_timing -output_format binary > my_timing_report.bin

# Load the binary report into the timing debugger
load_timing_debug_report my_timing_report.bin

# Highlight the specific path number passed by the user
highlight_timing_report -file my_timing_report.bin -path $path_num

puts "Successfully highlighted timing path number: $path_num"
}

innovus 22> source scripts/onboarding.tcl
innovus 23> highlight_path 1
-----
| NumPaths=1, Time=19074 microseconds |
-----
Parsing file my_timing_report.bin...
-----
| FileName=my_timing_report.bin, NumPaths=1, Time=340 microseconds |
-----
Successfully highlighted timing path number: 1

```

v. The command `get_db` is an extremely powerful command that gives you information about anything in the design. For example, the command

```
get_db insts
```

Returns a list of every instance in the design. As you can imagine, it's going to be a very long list. Some arguments also have attributes. For example, if I want a list of the pins of every instance, we can write

```
get_db insts .pins
```

If I want to know the library cell of some instances, we can write

```
get_db insts .lib_cell
```

Write a proc that uses `get_db` to return the fanout of every buffer in the clock tree.

It is recommended to use `get_db` in the terminal and press tab so to see a list of valid attributes.

```
proc dump_fanouts {} {  
    # Iterate through all instances in the design  
    foreach inst [get_db insts] {  
        # Get the library cell name  
        set cell_name [get_db $inst .base_cell.name]  
  
        # Check if the instance is one of our CTS clock buffers  
        if {$cell_name == "CLKBUFX2" || $cell_name == "CLKBUFX4" || $cell_name ==  
"CLKBUFX8"} {  
            # Get the output pin of the buffer  
            set out_pin [get_db $inst .pins -if {.direction == out}]  
  
            # Traverse to the connected net and count the number of endpoints (loads)  
            set fanout [get_db $out_pin .net.num_loads]  
  
            set inst_name [get_db $inst .name]  
  
            puts "Clock Buffer: $inst_name | Type: $cell_name | Fanout: $fanout"  
        }  
    }  
}
```

```

Clock Buffer: prects_FE_OCPC117_buffer_word_54 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC112_sram_a_do_14 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC111_sram_a_do_30 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC104_sram_b_do_27 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC102_sram_b_do_9 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC101_sram_b_do_10 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC100_sram_b_do_25 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC99_sram_b_do_7 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC98_sram_b_do_5 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC97_sram_b_do_24 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC96_sram_b_do_23 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC93_sram_a_do_4 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC91_sram_a_do_9 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC90_sram_a_do_2 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC88_sram_a_do_26 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC82_sram_a_do_25 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC81_sram_a_do_31 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC79_sram_a_do_19 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC78_sram_a_do_7 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC77_sram_a_do_8 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC76_sram_a_do_18 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC75_sram_a_do_11 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC74_sram_a_do_10 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC73_sram_a_do_29 | Type: CLKBUF2 | Fanout: 1
Clock Buffer: prects_FE_OCPC63_u_ctrl_r_ptr_2 | Type: CLKBUF4 | Fanout: 3

```

The `get_db` command acts as a pointer-based query system to traverse the Innovus design hierarchy.

1. I used `[get_db insts]` to grab a collection of every instance in the design.
2. I used `[get_db $inst .base_cell.name]` to traverse from the instance pointer up to its library definition to check if it was a clock buffer.
3. To find the pins, I used an inline filter: `[get_db $inst .pins -if {.direction == out}]` to instantly return only the output pin pointer.
4. To find the fanout, I traversed from the pin down to the physical wire using `[get_db $out_pin .net.num_loads]`, which retrieves the attribute counting the total number of endpoints on that net.

6. Static Timing Analysis on our 32-bit Adder

The questions in this section involve analyzing the timing reports generated by running the flowtool in Github repository on the 32-bit adder design.

You should have already run the flow once, and you will have to run it a few more times. In case you haven't run it yet, please complete

i. Identify the Worst Negative Slack (WNS) from the timing summary for both setup and hold in the current design.

Hint: There are multiple ways to find this information. The file *qor.html* may be a good place to start.

- Setup WNS: -1.793 ns
- Hold WNS: -0.593 ns

ii. Using the WNS and the clock frequency (set in the file *resources/sdcpipeline_src/base.sdc*), calculate the max frequency that the current design can theoretically run at.

Hint: Frequency is the reciprocal of period.

- Current Period = 1.0 ns
- Setup WNS = -1.793 ns
- Minimum viable period = 1.0 ns + 1.793 ns = 2.793 ns
- Max Frequency = $1 / 2.793 \text{ ns} \approx 358 \text{ MHz}$

iii. Modify the current design SDC file and rerun the flow to make the WNS at least +0.01ns. Include the screenshot of the flow summary or *qor.html* showing your positive WNS.

```
#####
# Generated by: Cadence Innovus 25.11-s102_1
# OS: Linux x86_64 (Host ID ece-rschrsv.ece.gatech.edu)
# Generated on: Fri Mar 6 00:16:35 2026
# Design: top_lvl
# Command: report_timing_summary -checks {setup drv} > [file join
#####
# SETUP WNS TNS FEP
#-----
View : ALL 0.050 0.000 0
Group : in2out N/A N/A 0
Group : reg2out N/A N/A 0
Group : in2reg 1.311 0.0 0
Group : reg2reg 0.050 0.0 0
#-----
```

To fix the negative slack and make the design pass timing, I modified the *top_level.sdc* file to relax the clock period from 1.0 ns to 3.0 ns. Because the minimum viable clock period calculated earlier was 2.793 ns, a 3.0 ns clock gives the signals enough time to propagate. After re-running the flow, the Setup Worst Negative Slack (WNS) improved to a positive 0.050 ns and the Total Negative Slack (TNS) is 0.000, confirming the design now successfully meets all timing requirements.

7. Investigating DRCs with the ChipFoundry SRAM

At this point, our design is meeting timing!

Does that mean our chip is ready for tape-out? No, unfortunately not.

This section will familiarize you with Design Rule Checking (DRC), another challenge we have to consider in the Physical Design process.

In this section, you will be implementing and understanding the importance of macros in VLSI design. Within your Verilog design, you instantiate SRAMs to use as memory blocks. These cannot be implemented with RTL; instead, we have to instantiate them as design macros.

For help using Innovus, see the Using Innovus guide in the docs/ directory.

i. Using the Cadence Support website, look for a command to be able to manually place these SRAM instances in the design.

In the box below, provide these commands. Explain why we might be manually placing them.

```
set_db [get_db insts *sram_A*] .location {150 480}
set_db [get_db insts *sram_B*] .location {550 480}
set_db [get_db insts *sram_*] .place_status fixed
```

We manually place the SRAM macros because the tool's default automated placement often pushes them into corners (like 50, 50) or against edges where they can block standard cell placement and create massive routing congestion. By manually defining their X and Y coordinates to be side-by-side in a more central location, we ensure they align correctly with the core grid and leave enough physical space around their pins for the routing tool to connect wires without causing thousands of short-circuit DRC violations. Fixing their status ensures subsequent steps don't move them.

ii. Do a run through to floorplan to verify that you've placed the SRAMs in the design. To save time, you can run `flowtool -verbose -from floorplan -to floorplan` and open that database.

NOTE: Unless you run `make clean` between flows, the outputs of your most recent flow will be in, for example `db/postroute.enc_2` (or higher if you have run more than 2 flows). If you run `make clean`, you will have to run through synthesis again. Keep that in mind!

Once you've verified that the macros have been placed in the floorplan, you can go ahead and run the flow all the way to postroute. Open the design in Innovus, and you'll see a bunch of white markers!

The white markers in the design are what are known as DRC violations. DRC stands for Design Rule Checking.

Foundries supply chip designers with design rules that specify requirements the design must meet for it to be taped out. Design libraries often have hundreds of rules. Some examples are minimum spacing between elements, layer overlap, and area limitations.

These are rules the foundry supplies us with allowing us to know whether a chip can actually be manufactured. Your goal as a PD engineer should be to create a manufacturable design, and in doing so you have to correct the DRC violations you've found. Provide an image of the design with the DRCs.



We will spend some time understanding why these DRCs occurred and brainstorming some possible fixes. Do to so, it's good to understand how Innovus sees these macros first.

Innovus instantiates these macros as a blackbox. It cannot see what's inside the macro. In this case, Innovus cannot see the bit cells, bit lines, sense amplifiers, etc. that an SRAM generally has. All innovus knows is the size of the blackbox, where its placed, and where the pins are, so Innovus can interface with the macro.

The ChipFoundry SRAM's pins are at the bottom of the macro. That's where all the DRCs are as well! We can reasonably infer that the cause of the DRCs come from how the pins are connected.

iii. Use Innovus to zoom in to a couple of the DRCs (can be done by scrolling) and click on a few of the markers. Innovus will show you the reasons for the DRCs. Paste or summarize what Innovus had to say about these DRCs.

Innovus primarily reports "Short" and "Min Spacing" violations for these specific markers. It indicates that Metal 1 (met1) wires are either directly overlapping where they shouldn't be, or they are routed too closely together, violating the foundry's minimum allowable distance between distinct metal tracks.

As you can see, Innovus doesn't have much to say about them. From here, knowing what specifically happened is a big jump, so we'll give you a hint!

iv. Open the file `flowtool/resources/CF_SRAM_1024x32/lef/CF_SRAM_1024x32.lef` in a text editor.

This LEF file describes the physical dimensions of the SRAM, the dimensions of the pins, and a bunch of other useful information/properties about the SRAM. Look through the LEF and indicate the metal layer that most of the pins are on in the macro.

Looking at the LEF file snippet, the signal pins (like DO[16]) are defined with `LAYER met1`. Therefore, most of the pins for this SRAM macro are located on **Metal 1 (met1)**.

v. Scroll to the very bottom of the file. Here, you will see an OBS block with a few metal layers within the block. What's the physical meaning of this block? Describe what this block does for each metal layer.

The OBS stands for **Obstruction**. This block tells the Innovus routing tool where it is physically not allowed to draw wires over the macro, in order to prevent short-circuiting with the SRAM's internal geometry.

- **For met1 and met2:** The obstruction rectangles (0.000 to 387.870 and 306.775) cover almost the entire bounding box of the SRAM. This means the router is forbidden from routing any wires over the macro using Metal 1 or Metal 2. (Note: The met1 blockage starts at Y=2.000, leaving a tiny 2µm sliver at the very bottom unblocked specifically so the pins can be accessed).
- **For met3:** The obstruction is only a tiny vertical sliver in the middle (X=193.770 to 194.1), meaning the router is mostly free to use Metal 3 to route wires completely over the top of the SRAM block.

vi. Use Innovus to find the inherent cause of the DRC violations by looking at how the tool decided to connect the nets to the pins. Describe what causes these DRC violations and prove it with a screenshot of what's going on. Remember what layer most of the pins are on.

The inherent cause of the DRCs is a lack of physical routing space. The SRAM's pins are densely packed on met1 at the bottom edge. However, the OBS block forbids any routing on met1 or met2 directly inside the macro area. Because we initially did not have a keep-out margin, standard cells were placed flush against the bottom of the SRAM. The router was trapped: it couldn't route "up" because of the met1/met2 obstruction, and it couldn't route "down" without immediately crashing into the standard cells' own met1 pins, resulting in massive short-circuits and spacing violations.

vii. Propose a solution to fix the DRCs. Don't overthink it. Remember what the OBS block defines for each metal layer.

We need to give the routing tool "breathing room" to access those dense met1 pins. We should implement a "keep-out margin" (or placement halo) around the SRAM macro. This will forbid standard cells from being placed immediately next to the SRAM, giving the router the physical empty space it needs to drop down from higher metal layers (like met3 or met4) into the met1 pins without violating obstructions or shorting with standard cells.

viii. To fix these DRC violations, you will need to make two changes:

1. Make the appropriate change (just 1 change) in setup.yaml to fix the DRC violations. Once again, remember the layer most of the pins are on in the LEF and why the DRCs occurred. Most importantly, use the solution in part 7 to help decide what change you need to make.

2. This second one is a bit hard to figure out by yourselves right now, so here's a hint: You will need to define a keep-out margin around the SRAMs such that no standard cells can be placed there. Find a command on Cadence Support that does this and include it in floorplan.tcl.

1. In the setup.yaml file, the routing_layers: bottom attribute was originally set to met2. This restricted the routing tool from using Metal 1 for any global or detailed routing. Since the ChipFoundry SRAM pins are located on met1, the tool was physically unable to connect to them without causing DRC violations. By changing the bottom routing layer to met1, we allow the router to access the necessary metal layer to interface with the macro pins.

```
2. foreach my_macro [get_db insts *sram_*] {  
    create_place_halo -insts $my_macro -halo_deltas {5.0 5.0 5.0 5.0}  
}
```

lv. State the changes you made and paste a picture of your DRC clean design!

Implemented Fixes:

- **Manual Placement:** SRAMs placed at {150 480} and {550 480} to prevent corner-clumping.
- **Placement Halo:** Added a 5.0µm keep-out margin via create_place_halo to resolve pin congestion.
- **Layer Config:** Updated setup.yaml to set the bottom routing layer to met1, enabling physical connection to the SRAM pins.

This is actually one of the problems we fixed very recently! If you think this section felt a bit pedantic, that's because we went through this exact process to figure out how to get a DRC clean design. The point is to expose to you an example of what we regularly do.